

# LAB1 실험 후 보고서

조합논리 10종 — AND/OR/XOR 게이트부터 7세그먼트 디코더까지 · Vivado 2026.1 시뮬레이션·합성·구현·보드 검증 결과

이름

형종희

학번

2023440145

조 번호

C조

작성일

2026년 9월 27일

## 요약

LAB1의 조합논리 회로 10종 전부에 대해 Vivado Behavioral Simulation → Synthesis → Implementation(DRC) → Bitstream 생성 → 실제 보드(Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드) 프로그래밍 및 동작 확인까지 모두 완료했다. 10개 회로 모두 시뮬레이션 통과, DRC 위반 0건, 보드 동작이 예상 진리표와 일치함을 확인했다.

## 제출 저장소 정보

소스·테스트벤치·XDC·보드 검증 사진/영상·본 보고서 전체를 아래 개인 GitHub 저장소에 커밋하고 제출 태그를 남겼다.

|       |                                                                                                                     |
|-------|---------------------------------------------------------------------------------------------------------------------|
| 저장소   | <a href="https://github.com/hyeongjonghee/electronics2-LAB1">https://github.com/hyeongjonghee/electronics2-LAB1</a> |
| 제출 태그 | lab1-submit                                                                                                         |
| 커밋 해시 | 3be4c53                                                                                                             |

## 목차

- [1. 제출 저장소 정보](#)
- [2. LAB1-01 · AND·OR·XOR 논리 게이트](#)
- [3. LAB1-02 · 전가산기 \(Full Adder\)](#)
- [4. LAB1-03 · 4비트 가산기 \(Adder-4\)](#)
- [5. LAB1-04 · 4비트 감산기 \(Subtractor-4\)](#)

6. [LAB1-05 · 4비트 크기 비교기 \(Comparator-4\)](#)
7. [LAB1-06 · 4:1 멀티플렉서 \(MUX 4-to-1\)](#)
8. [LAB1-07 · 1:8 디멀티플렉서 \(DEMUX 1-to-8\)](#)
9. [LAB1-08 · 8:3 인코더 \(Encoder 8-to-3\)](#)
10. [LAB1-09 · 3:8 디코더 \(Decoder 3-to-8\)](#)
11. [LAB1-10 · 7세그먼트 디코더 \(Seven-Segment Decoder\)](#)
12. [실험 중 겪은 문제와 해결](#)
13. [합성 자원 사용량 종합](#)
14. [종합 결론 및 제출 체크리스트](#)

## LAB1-01 AND·OR·XOR 논리 게이트 (logic\_gate)

### 1. 회로 개요

목적: 입력 a, b 두 비트로 AND(x)·OR(y)·XOR(z) 세 가지 조합논리 결과를 동시에 만든다.

포트 구성

| 구분 | 신호명     | 비트 폭                  |
|----|---------|-----------------------|
| 입력 | a, b    | 1비트                   |
| 출력 | x, y, z | 1비트 (각각 AND, OR, XOR) |

예상 진리표

| a | b | x (AND) | y (OR) | z (XOR) |
|---|---|---------|--------|---------|
| 0 | 0 | 0       | 0      | 0       |
| 0 | 1 | 0       | 1      | 1       |
| 1 | 0 | 0       | 1      | 1       |
| 1 | 1 | 1       | 1      | 0       |

### 2. RTL 코드

Verilog · logic\_gate.v

```
module logic_gate(input wire a,b, output wire x,y,z);
    assign x = a & b;
    assign y = a | b;
    assign z = a ^ b;
endmodule
```

### 3. Vivado Behavioral Simulation 결과

✅ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS logic\_gate cases=4

검사 항목 4개 전부 통과, 시뮬레이션 종료 시각 40ns — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

#### 4. 합성(Synthesis) 결과

##### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 2개**, **Bonded IOB 5개** 사용, Critical Warning 없음.

#### 5. 구현(Implementation) · DRC 결과

##### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 위반(violation) **0건**. Spartan-7 बैं크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

#### 6. 비트스트림 생성 및 보드 검증

##### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. 입력 a, b는 보드 버튼(PB)으로, 출력 x·y·z는 LED로 확인했다.

버튼 조합 4가지(00,01,10,11)를 모두 눌러 AND·OR·XOR LED가 예상 진리표대로 켜지는 것을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

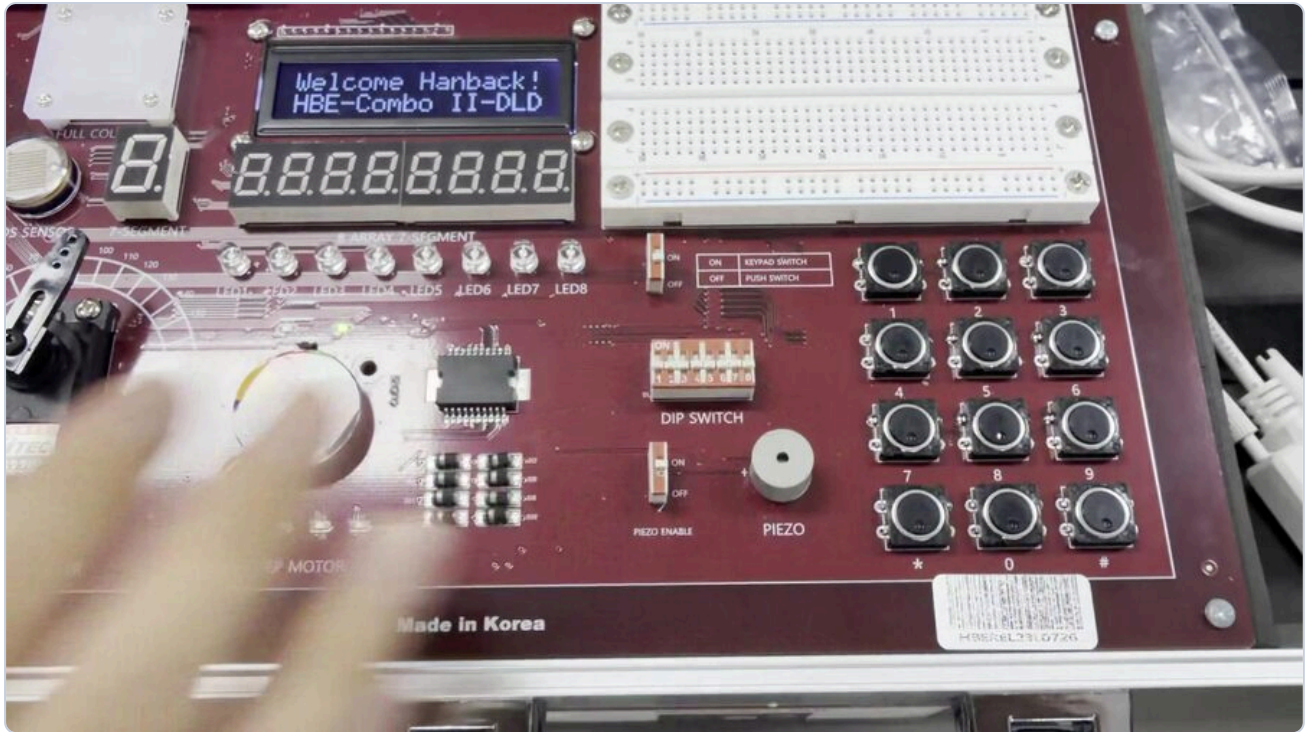


사진 01. LAB1-01 AND·OR·XOR 논리 게이트 보드 동작 확인 (입력 a, b는 보드 버튼(PB)으로, 출력 x·y·z는 LED로 확인했다.)

관련 영상: evidence/videos/LAB1-01\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## LAB1-02 전가산기 (Full Adder) (full\_adder)

### 1. 회로 개요

**목적:** 반가산기 두 개로 세 입력(a, b, cin)을 더한다. 첫 번째 반가산기의 합과 cin을 다시 더하고, 두 반가산기의 자리올림을 OR로 합친다.

포트 구성

| 구분 | 신호명       | 비트 폭 |
|----|-----------|------|
| 입력 | a, b, cin | 1비트  |
| 출력 | s, cout   | 1비트  |

예상 진리표

| a | b | cin | s | cout |
|---|---|-----|---|------|
| 0 | 0 | 0   | 0 | 0    |
| 0 | 1 | 1   | 0 | 1    |
| 1 | 1 | 1   | 1 | 1    |

### 2. RTL 코드

```
Verilog · full_adder.v

module full_adder(input wire a,b,cin, output wire s,cout);
  wire partial_sum, carry_ab, carry_cin;
  half_adder first(.a(a), .b(b), .s(partial_sum), .c(carry_ab));
  half_adder second(.a(partial_sum), .b(cin), .s(s), .c(carry_cin));
  assign cout = carry_ab | carry_cin;
endmodule
```

### 3. Vivado Behavioral Simulation 결과

✅ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS full\_adder cases=8

검사 항목 8개 전부 통과, 시뮬레이션 종료 시각 80ns — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

#### 4. 합성(Synthesis) 결과

##### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 1개**, **Bonded IOB 5개** 사용, Critical Warning 없음.

#### 5. 구현(Implementation) · DRC 결과

##### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 위반(violation) **0건**. Spartan-7 बैं크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

#### 6. 비트스트림 생성 및 보드 검증

##### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. DIP 스위치 3개로 a, b, cin을 입력하고 LED 2개로 s, cout을 확인했다.

8가지 입력 조합을 모두 넣어 LED로 표시되는 합(s)과 자리올림(cout)이 진리표와 일치함을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

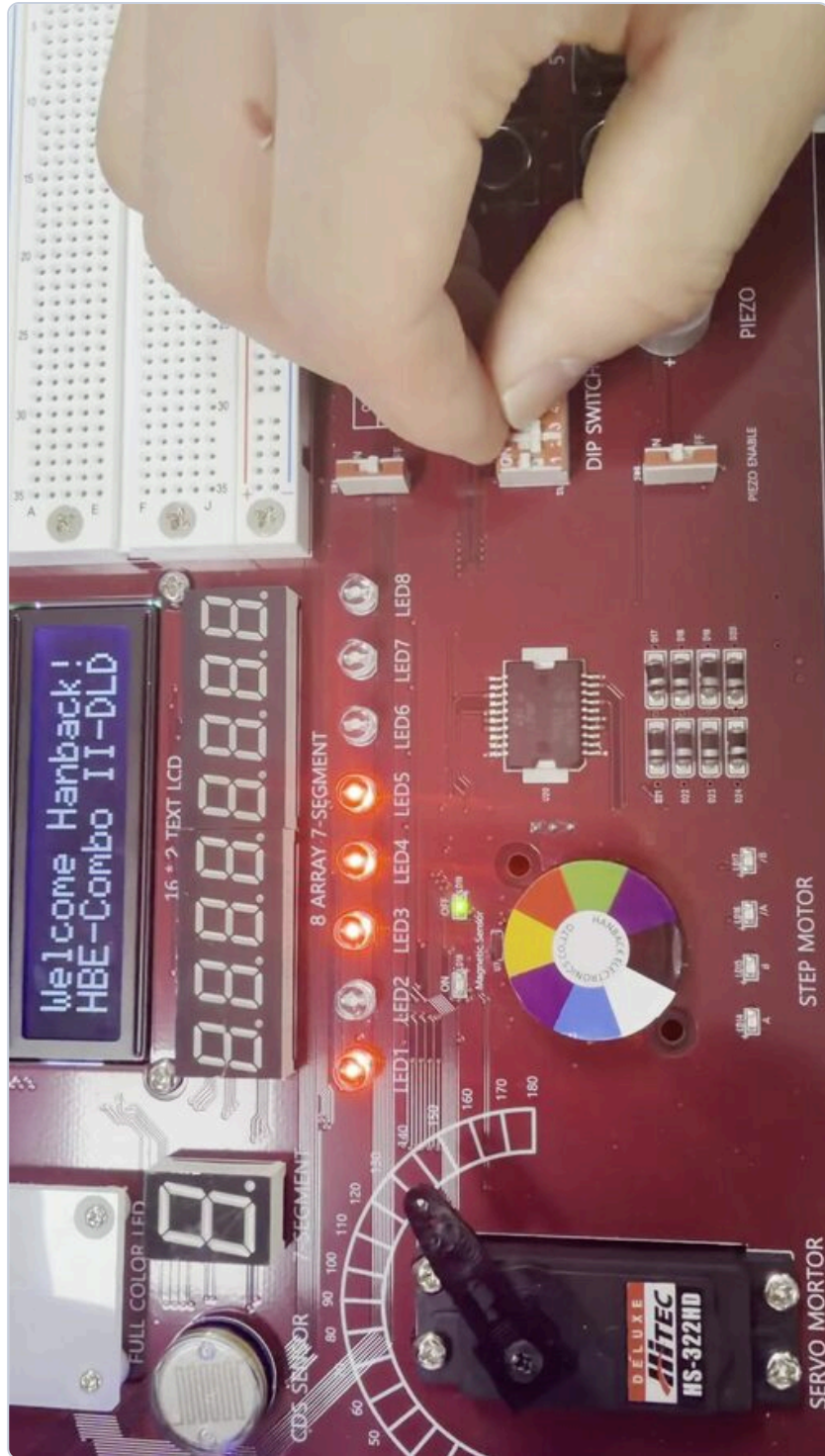


사진 02. LAB1-02 전가산기 (Full Adder) 보드 동작 확인 (DIP 스위치 3개로 a, b, cin을 입력하고 LED 2개로 s, cout을 확인했다.)

관련 영상: evidence/videos/LAB1-02\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## LAB1-03 4비트 가산기 (Adder-4) (adder\_4bit)

### 1. 회로 개요

목적: 0~15 범위의 두 수를 더한다. 결과는 carry 1비트와 합 4비트로 총 5비트다.

포트 구성

| 구분 | 신호명            | 비트 폭      |
|----|----------------|-----------|
| 입력 | a[3:0], b[3:0] | 4비트       |
| 출력 | s[3:0], cout   | 4비트 + 1비트 |

예상 진리표

| 입력      | 예상 결과        |
|---------|--------------|
| 0 + 0   | cout=0, s=0  |
| 1 + 15  | cout=1, s=0  |
| 15 + 15 | cout=1, s=14 |

### 2. RTL 코드

```
Verilog · adder_4bit.v

module adder_4bit(input wire [3:0] a,b, output wire [3:0] s, output wire cout);
    assign {cout,s} = {1'b0,a} + {1'b0,b};
endmodule
```

### 3. Vivado Behavioral Simulation 결과

✅ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS adder\_4bit cases=256

검사 항목 **256개** 전부 통과, 시뮬레이션 종료 시각 **2560ns** — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

#### 4. 합성(Synthesis) 결과

##### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 4개**, **Bonded IOB 13개** 사용, Critical Warning 없음.

#### 5. 구현(Implementation) · DRC 결과

##### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 위반(violation) **0건**. Spartan-7 뱅크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

#### 6. 비트스트림 생성 및 보드 검증

##### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. DIP 스위치 8개로 a[3:0], b[3:0]을 입력하고 LED 5개로 cout, s[3:0]을 확인했다.

대표 조합(0+0, 1+15, 15+15 등)을 DIP로 입력해 LED 합·캐리 값이 예상과 일치함을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

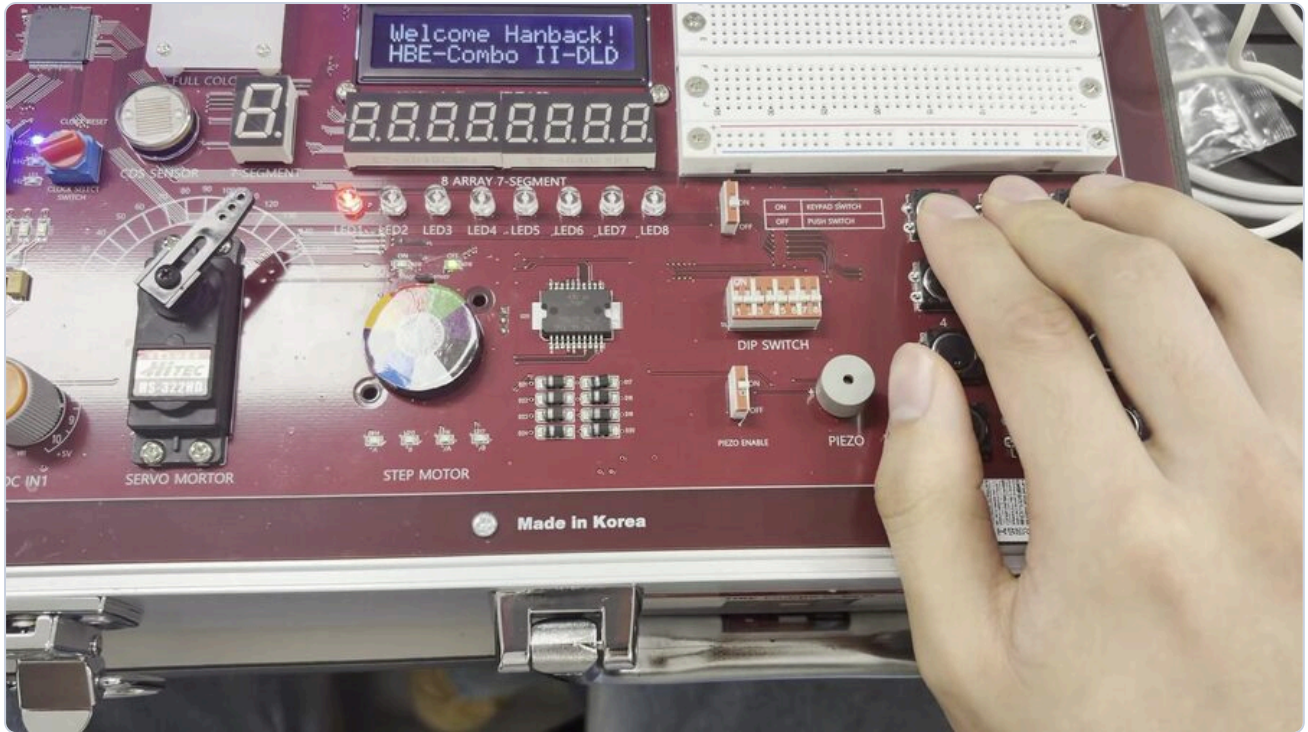


사진 03. LAB1-03 4비트 가산기 (Adder-4) 보드 동작 확인 (DIP 스위치 8개로 a[3:0], b[3:0]을 입력하고 LED 5개로 cout, s[3:0]을 확인했다.)

관련 영상: evidence/videos/LAB1-03\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## LAB1-04 4비트 감산기 (Subtractor-4) (sub\_4bit)

### 1. 회로 개요

**목적:** d는 a-b의 하위 4비트이고 bor는  $a < b$ 일 때만 1이다. 같은 수끼리 빼면 borrow는 0이다.

포트 구성

| 구분 | 신호명            | 비트 폭      |
|----|----------------|-----------|
| 입력 | a[3:0], b[3:0] | 4비트       |
| 출력 | d[3:0], bor    | 4비트 + 1비트 |

예상 진리표

| 입력     | 예상 결과       |
|--------|-------------|
| 0 - 0  | bor=0, d=0  |
| 0 - 1  | bor=1, d=15 |
| 15 - 0 | bor=0, d=15 |

### 2. RTL 코드

Verilog · sub\_4bit.v

```
module sub_4bit(input wire [3:0] a,b, output wire [3:0] d, output wire bor);
    assign d = a - b;
    assign bor = a < b; // Equal operands do NOT borrow.
endmodule
```

### 3. Vivado Behavioral Simulation 결과

✅ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS sub\_4bit cases=256

검사 항목 **256개** 전부 통과, 시뮬레이션 종료 시각 **2560ns** — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

#### 4. 합성(Synthesis) 결과

##### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 4개**, **Bonded IOB 13개** 사용, Critical Warning 없음.

#### 5. 구현(Implementation) · DRC 결과

##### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 위반(violation) **0건**. Spartan-7 뱅크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

#### 6. 비트스트림 생성 및 보드 검증

##### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. DIP 스위치 8개로 a[3:0], b[3:0]을 입력하고 LED 5개로 bor, d[3:0]을 확인했다.

0-1처럼 빌림이 발생하는 조합과 발생하지 않는 조합을 각각 넣어 bor LED와 d 값이 예상과 일치함을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

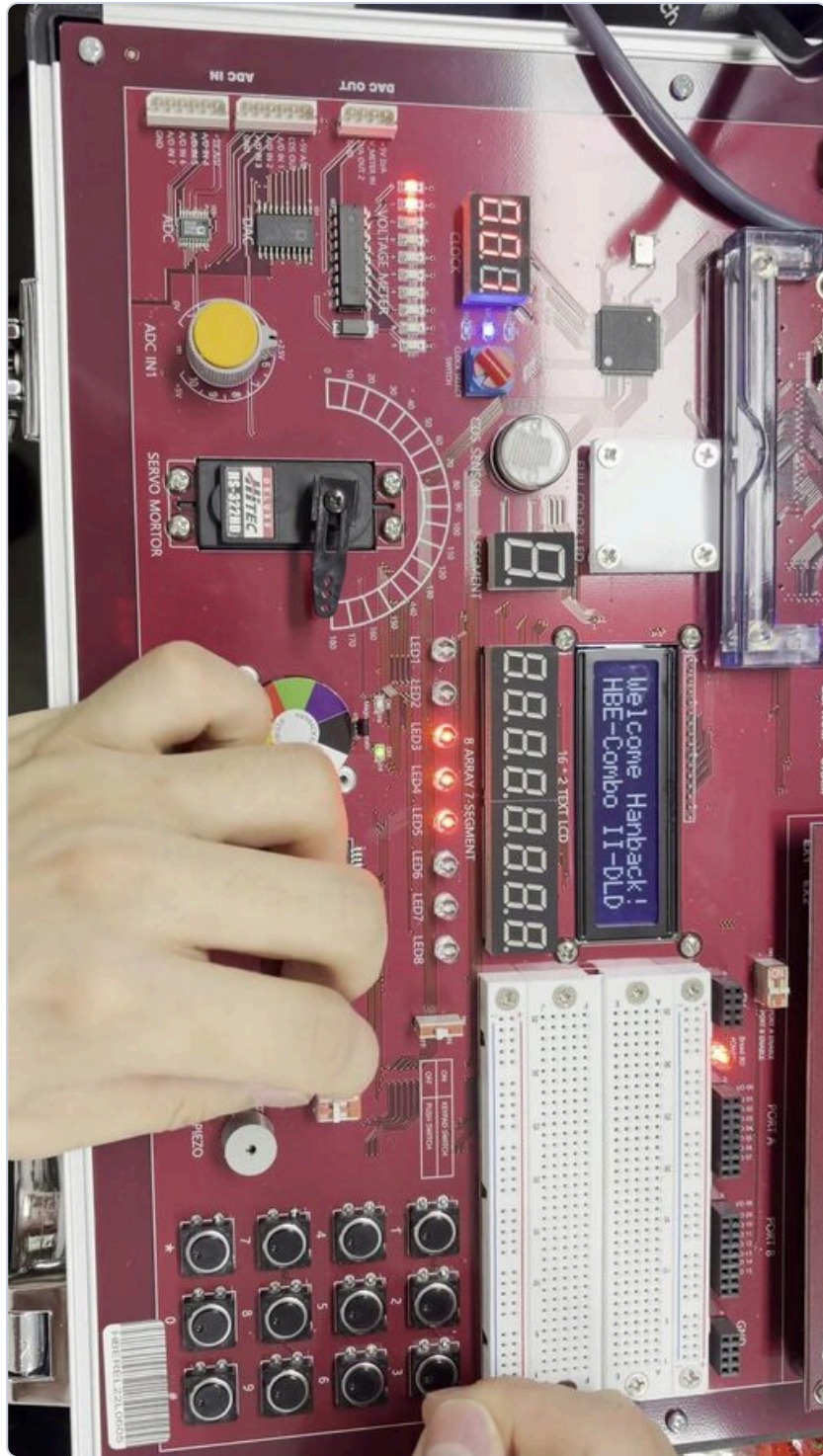


사진 04. LAB1-04 4비트 감산기 (Subtractor-4) 보드 동작 확인 (DIP 스위치 8개로 a[3:0], b[3:0]을 입력하고 LED 5개로 bor, d[3:0]을 확인했다.)

관련 영상: evidence/videos/LAB1-04\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## LAB1-05 4비트 크기 비교기 (Comparator-4) (compare\_4)

### 1. 회로 개요

목적:  $o[2]$ 는  $a > b$ ,  $o[1]$ 은  $a = b$ ,  $o[0]$ 은  $a < b$ 를 나타낸다. 세 결과 중 하나만 1이다.

포트 구성

| 구분 | 신호명                 | 비트 폭      |
|----|---------------------|-----------|
| 입력 | $a[3:0]$ , $b[3:0]$ | 4비트       |
| 출력 | $o[2:0]$            | 3비트 (원-핫) |

예상 진리표

| 입력    | 예상 결과( $o[2:0]$ ) |
|-------|-------------------|
| 0 대 0 | 010 (같음)          |
| 0 대 1 | 001 (작음)          |
| 1 대 0 | 100 (큼)           |

### 2. RTL 코드

```
Verilog · compare_4.v
module compare_4(input wire [3:0] a,b, output wire [2:0] o);
    assign o = {a > b, a == b, a < b};
endmodule
```

### 3. Vivado Behavioral Simulation 결과

✅ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS compare\_4 cases=256

검사 항목 **256개** 전부 통과, 시뮬레이션 종료 시각 **2560ns** — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

#### 4. 합성(Synthesis) 결과

##### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 5개**, **Bonded IOB 11개** 사용, Critical Warning 없음.

#### 5. 구현(Implementation) · DRC 결과

##### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 위반(violation) **0건**. Spartan-7 뱅크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

#### 6. 비트스트림 생성 및 보드 검증

##### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. DIP 스위치 8개로 `a[3:0]`, `b[3:0]`을 입력하고 LED 3개(큼/같음/작음)로 결과를 확인했다.

$a > b$ ,  $a = b$ ,  $a < b$  세 경우를 각각 입력해 매번 LED 한 개만 켜지고 그 위치가 예상과 일치함을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

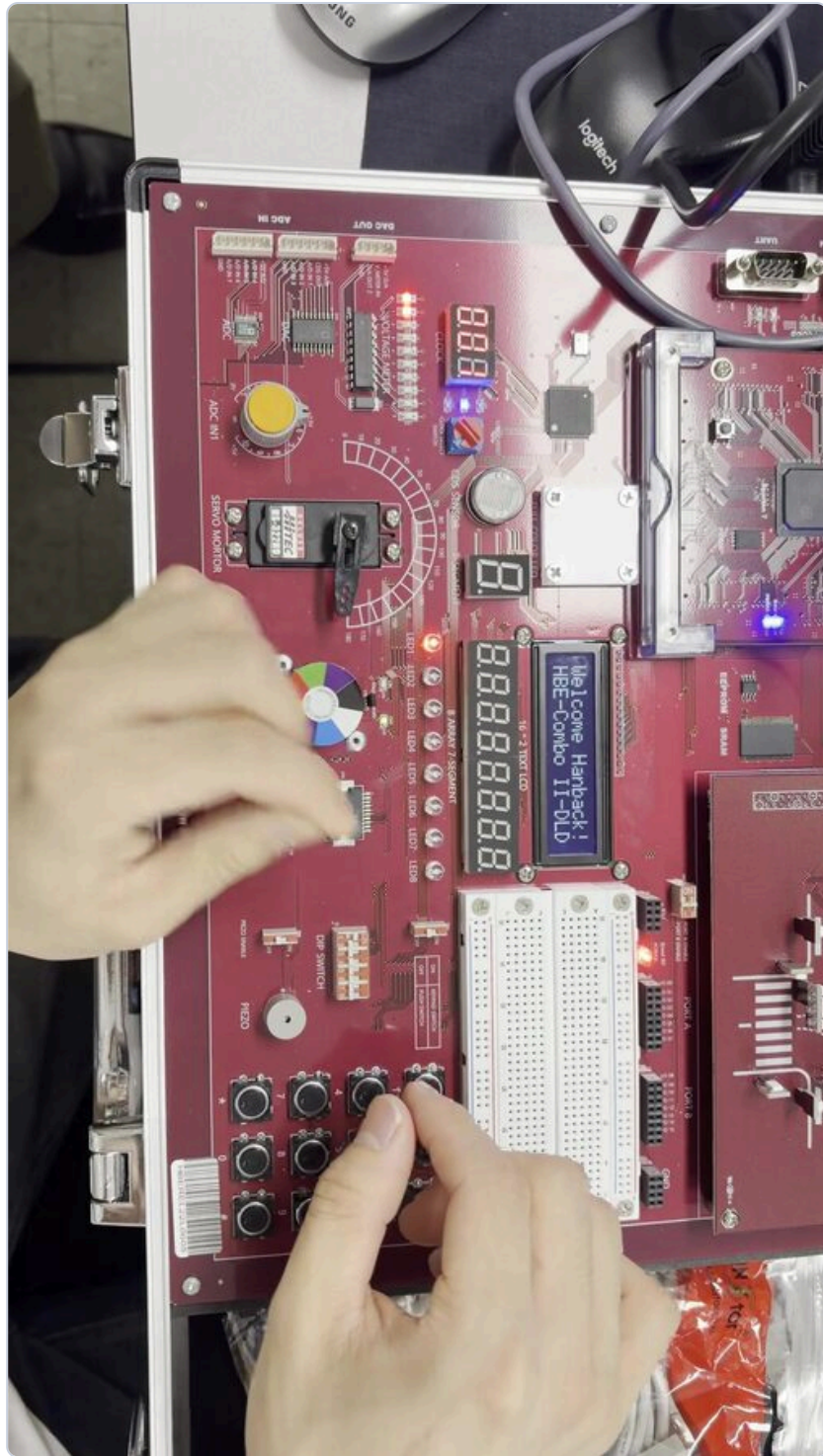


사진 05. LAB1-05 4비트 크기 비교기 (Comparator-4) 보드 동작 확인 (DIP 스위치 8개로 a[3:0], b[3:0]을 입력하고 LED 3개(큼/같음/작음)로 결과를 확인했다.)

관련 영상: evidence/videos/LAB1-05\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## LAB1-06 4:1 멀티플렉서 (MUX 4-to-1) (mux\_4x1)

### 1. 회로 개요

목적: 선택 s=00,01,10,11은 각각 i[3],i[2],i[1],i[0]을 z로 보낸다.

포트 구성

| 구분 | 신호명            | 비트 폭      |
|----|----------------|-----------|
| 입력 | i[3:0], s[1:0] | 4비트 + 2비트 |
| 출력 | z              | 1비트       |

예상 진리표

| 입력           | 예상 결과 |
|--------------|-------|
| i=1000, s=00 | z=1   |
| i=1000, s=01 | z=0   |
| i=0001, s=11 | z=1   |

### 2. RTL 코드

Verilog · mux\_4x1.v

```
module mux_4x1(input wire [3:0] i, input wire [1:0] s, output wire z);  
    // Preserve the original lab ordering: 00 -> i[3].  
    assign z = i[3-s];  
endmodule
```

### 3. Vivado Behavioral Simulation 결과

✅ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS mux\_4x1 cases=64

검사 항목 **64개** 전부 통과, 시뮬레이션 종료 시각 **640ns** — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

#### 4. 합성(Synthesis) 결과

##### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 1개**, **Bonded IOB 7개** 사용, Critical Warning 없음.

#### 5. 구현(Implementation) · DRC 결과

##### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 위반(violation) **0건**. Spartan-7 बैं크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

#### 6. 비트스트림 생성 및 보드 검증

##### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. 버튼 4개로 `i[3:0]`, DIP 스위치 2개로 `s[1:0]`을 입력하고 LED 1개로 `z`를 확인했다.

`s`를 00 → 11까지 바꿔가며 각 선택 입력이 그대로 `z`에 나타나는 것을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

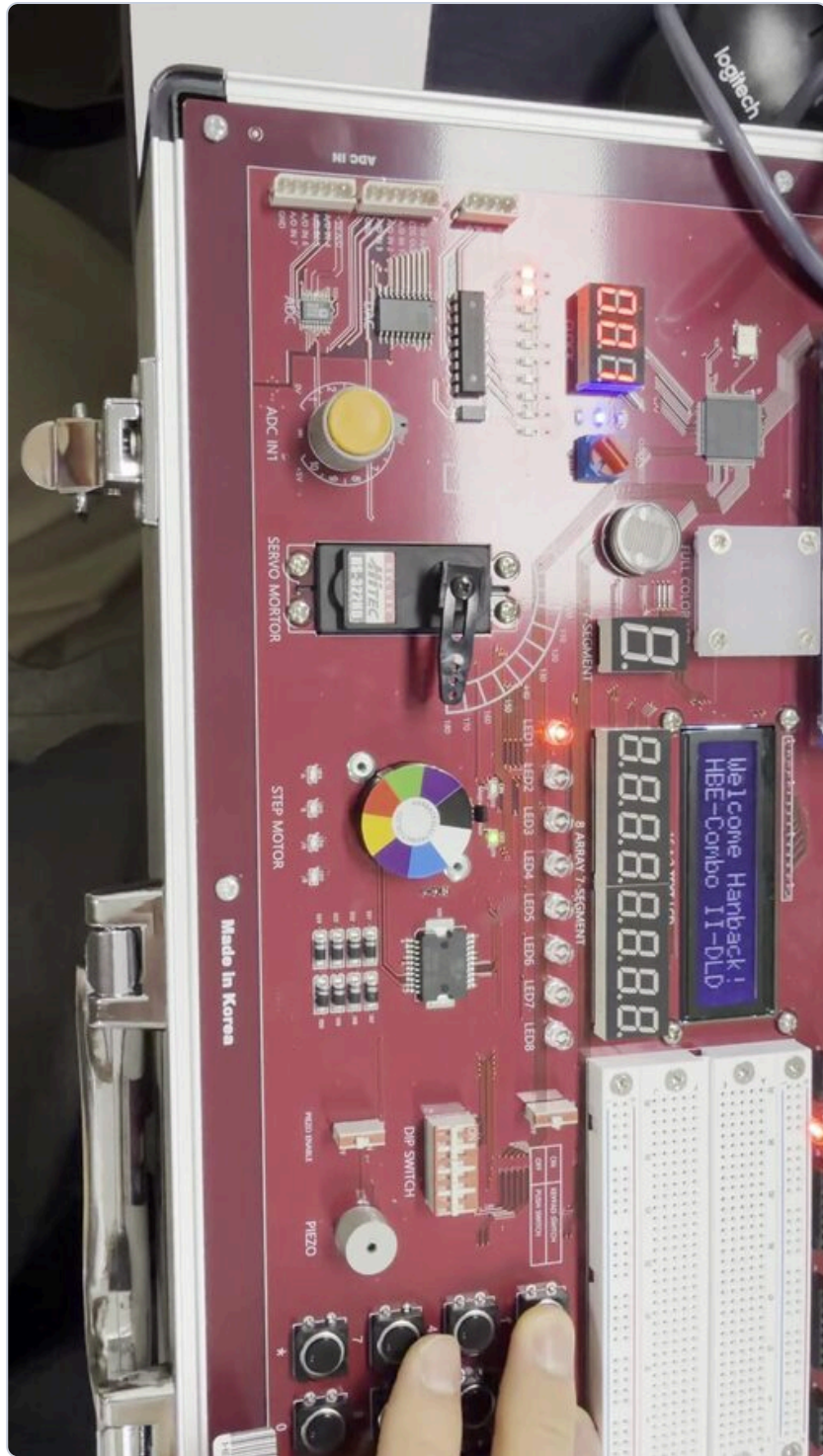


사진 06. LAB1-06 4:1 멀티플렉서 (MUX 4-to-1) 보드 동작 확인 (버튼 4개로  $i[3:0]$ , DIP 스위치 2개로  $s[1:0]$ 을 입력하고 LED 1개로  $z$ 를 확인했다.)

관련 영상: evidence/videos/LAB1-06\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## LAB1-07 1:8 디멀티플렉서 (DEMUX 1-to-8) (demux\_1x8)

### 1. 회로 개요

**목적:** 입력  $i$ 를 선택된 한 출력에 보낸다.  $s=000$ 이면  $o[7]$ ,  $s=111$ 이면  $o[0]$ 이다.  $i=0$ 이면 모든 출력이 0이다.

#### 포트 구성

| 구분 | 신호명         | 비트 폭      |
|----|-------------|-----------|
| 입력 | $i, s[2:0]$ | 1비트 + 3비트 |
| 출력 | $o[7:0]$    | 8비트       |

#### 예상 진리표

| 입력             | 예상 결과    |
|----------------|----------|
| $i=0$ , 모든 $s$ | 00000000 |
| $i=1, s=000$   | 10000000 |
| $i=1, s=111$   | 00000001 |

### 2. RTL 코드

Verilog · demux\_1x8.v

```
module demux_1x8(input wire i, input wire [2:0] s, output wire [7:0] o);
    // Preserve the original lab ordering: 000 -> o[7].
    assign o = i ? (8'b10000000 >> s) : 8'b0;
endmodule
```

### 3. Vivado Behavioral Simulation 결과

#### ✅ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS demux\_1x8 cases=16

검사 항목 **16개** 전부 통과, 시뮬레이션 종료 시각 **160ns** — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

#### 4. 합성(Synthesis) 결과

##### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 4개**, **Bonded IOB 12개** 사용, Critical Warning 없음.

#### 5. 구현(Implementation) · DRC 결과

##### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 위반(violation) **0건**. Spartan-7 뱅크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

#### 6. 비트스트림 생성 및 보드 검증

##### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. 버튼 1개로 `i`, DIP 스위치 3개로 `s[2:0]`을 입력하고 LED 8개로 `o[7:0]`을 확인했다.

`s`를 바꿔가며 LED 8개 중 정확히 한 개(또는 `i=0`일 때 전부 소등)만 켜지는 것을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

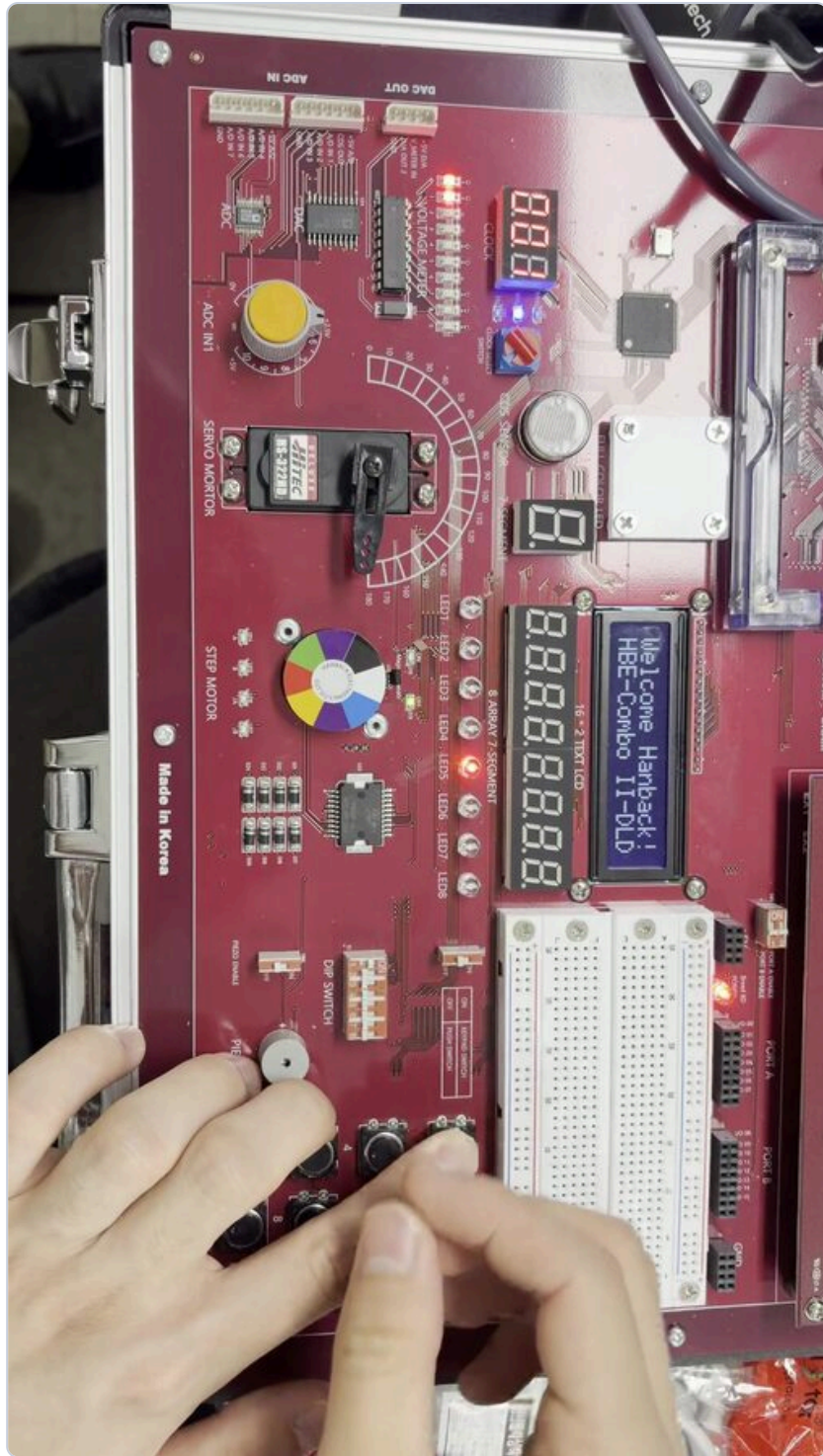


사진 07. LAB1-07 1:8 디멀티플렉서 (DEMUX 1-to-8) 보드 동작 확인 (버튼 1개로 i, DIP 스위치 3개로 s[2:0]을 입력하고 LED 8개로 o[7:0]을 확인했다.)

관련 영상: evidence/videos/LAB1-07\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## LAB1-08 8:3 인코더 (Encoder 8-to-3) (encoder8x3)

### 1. 회로 개요

**목적:** 한 비트만 켜진 입력을 번호로 바꾼다. i[7]만 1이면 0, i[0]만 1이면 7이다. 0 또는 다중 입력은 000을 출력한다.

포트 구성

| 구분 | 신호명    | 비트 폭 |
|----|--------|------|
| 입력 | i[7:0] | 8비트  |
| 출력 | a[2:0] | 3비트  |

예상 진리표

| 입력       | 예상 결과        |
|----------|--------------|
| 10000000 | 000          |
| 00000001 | 111          |
| 00000011 | 000 (잘못된 입력) |

### 2. RTL 코드

Verilog · encoder8x3.v

```
module encoder8x3(input wire [7:0] i, output reg [2:0] a);
    // One-hot only. Zero and multi-hot inputs return 000 (not a priority encoder).
    always @* begin
        case (i)
            8'h80: a=0; 8'h40: a=1; 8'h20: a=2; 8'h10: a=3;
            8'h08: a=4; 8'h04: a=5; 8'h02: a=6; 8'h01: a=7;
            default: a=0;
        endcase
    end
endmodule
```

### 3. Vivado Behavioral Simulation 결과

#### ✓ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS encoder8x3 cases=256

검사 항목 **256개** 전부 통과, 시뮬레이션 종료 시각 **2560ns** — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

### 4. 합성(Synthesis) 결과

#### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 5개**, **Bonded IOB 11개** 사용, Critical Warning 없음.

### 5. 구현(Implementation) · DRC 결과

#### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 **위반(violation) 0건**. Spartan-7 बैं크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

### 6. 비트스트림 생성 및 보드 검증

#### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. 버튼 8개로 `i[7:0]`을 입력하고 LED 3개로 `a[2:0]`을 확인했다.

한 번에 버튼 하나만 누른 one-hot 조합들에 대해 LED가 이진수로 번호를 표시하는 것을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

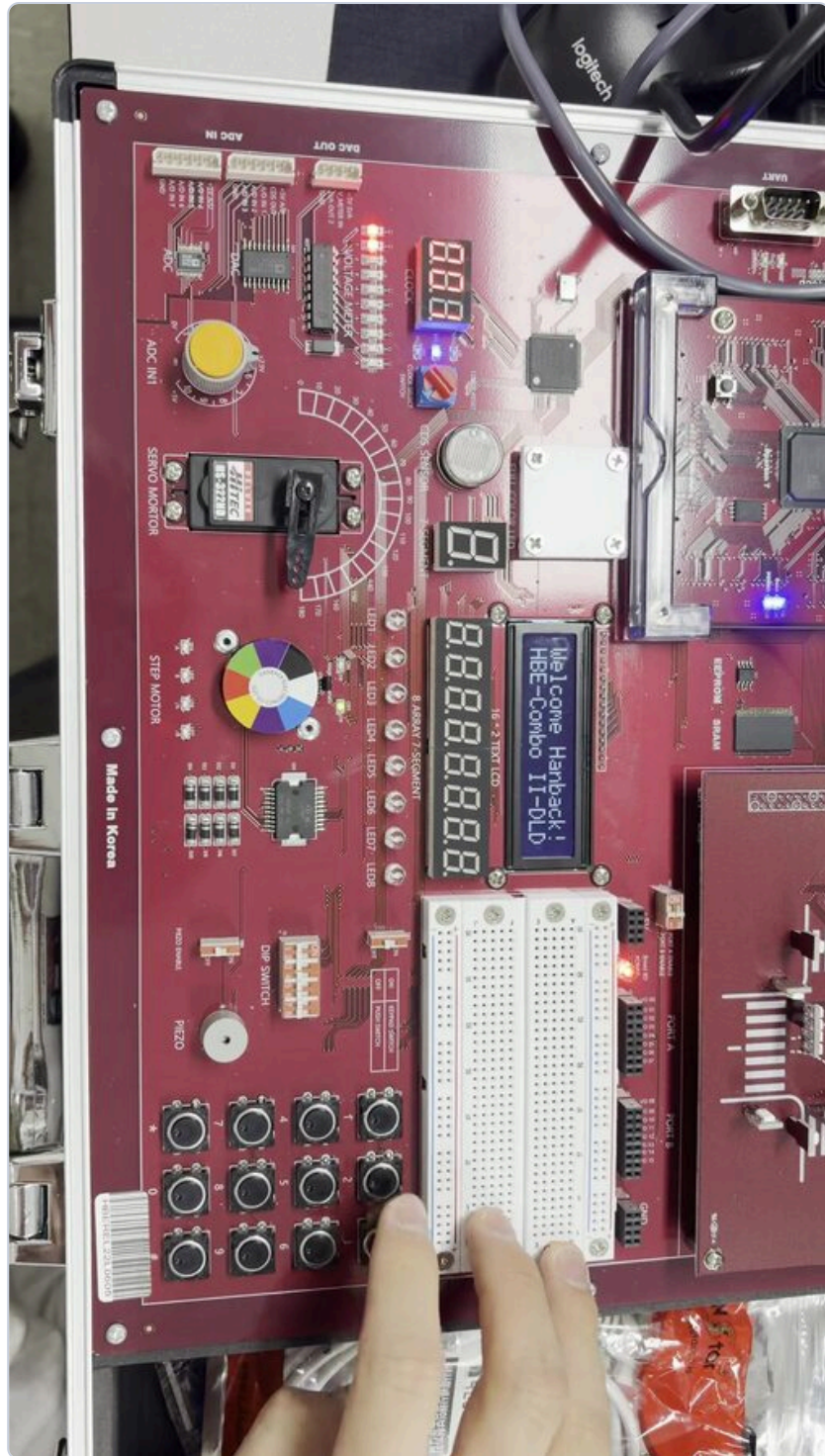


사진 08. LAB1-08 8:3 인코더 (Encoder 8-to-3) 보드 동작 확인 (버튼 8개로 i[7:0]을 입력하고 LED 3개로 a[2:0]을 확인했다.)

관련 영상: evidence/videos/LAB1-08\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## LAB1-09 3:8 디코더 (Decoder 3-to-8) (decoder3x8)

### 1. 회로 개요

목적: 입력 abc의 이진 번호와 같은 o 비트를 1로 만든다. 000은 o[0], 111은 o[7]이다.

포트 구성

| 구분 | 신호명     | 비트 폭      |
|----|---------|-----------|
| 입력 | a, b, c | 1비트씩 3개   |
| 출력 | o[7:0]  | 8비트 (원-핫) |

예상 진리표

| 입력  | 예상 결과    |
|-----|----------|
| 000 | 00000001 |
| 011 | 00001000 |
| 111 | 10000000 |

### 2. RTL 코드

```
Verilog · decoder3x8.v
module decoder3x8(input wire a,b,c, output wire [7:0] o);
    assign o = 8'b00000001 << {a,b,c};
endmodule
```

### 3. Vivado Behavioral Simulation 결과

✅ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS decoder3x8 cases=8

검사 항목 8개 전부 통과, 시뮬레이션 종료 시각 80ns — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

#### 4. 합성(Synthesis) 결과

##### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 4개**, **Bonded IOB 11개** 사용, Critical Warning 없음.

#### 5. 구현(Implementation) · DRC 결과

##### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 위반(violation) **0건**. Spartan-7 뱅크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

#### 6. 비트스트림 생성 및 보드 검증

##### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. 버튼 3개로 a, b, c를 입력하고 LED 8개로 o[7:0]을 확인했다.

000부터 111까지 8가지 입력을 모두 넣어 LED 8개 중 정확히 한 개만 켜지고 그 위치가 이진값과 일치함을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

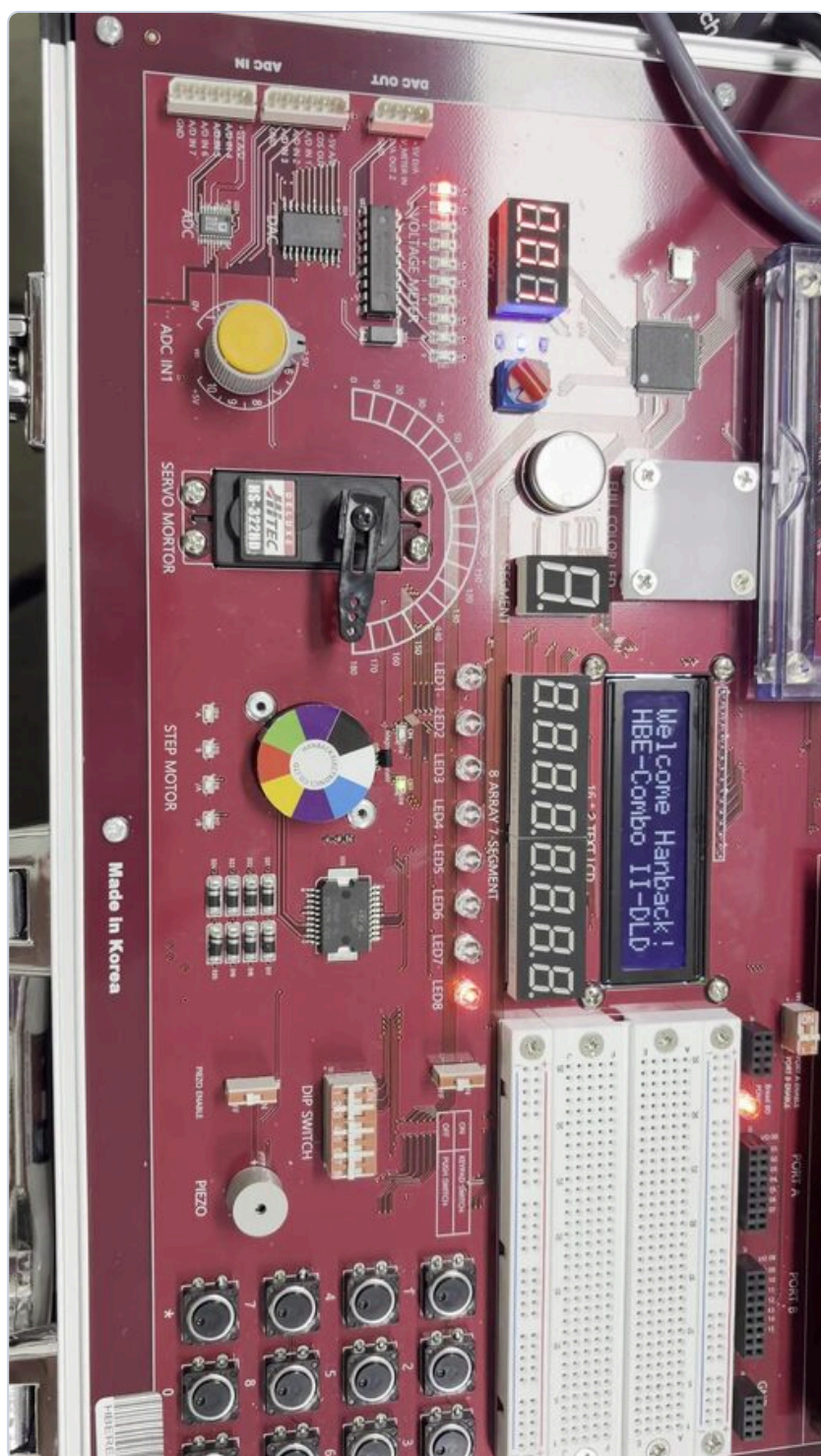


사진 09. LAB1-09 3:8 디코더 (Decoder 3-to-8) 보드 동작 확인 (버튼 3개로 a, b, c를 입력하고 LED 8개로 o[7:0]을 확인했다.)

관련 영상: [evidence/videos/LAB1-09\\_\\*.mp4](#) ([GitHub 저장소](#)에서 확인 가능)

## LAB1-10 7세그먼트 디코더 (Seven-Segment Decoder) (seg\_decoder)

### 1. 회로 개요

**목적:** 4비트 입력의 0~F를 표시한다. seg\_data[7:0] 순서는 a,b,c,d,e,f,g,dp이며 1에서 점등하고 dp는 항상 0이다.

#### 포트 구성

| 구분 | 신호명           | 비트 폭           |
|----|---------------|----------------|
| 입력 | bcd[3:0]      | 4비트 (0~15)     |
| 출력 | seg_data[7:0] | 8비트, a~g,dp 순서 |

#### 예상 진리표

| 입력(16진수) | 예상 결과 | 켜지는 획       |
|----------|-------|-------------|
| 0        | FC    | a,b,c,d,e,f |
| 1        | 60    | b,c         |
| 8        | FE    | a-g 전부      |
| F        | 8E    | a,e,f,g     |

### 2. RTL 코드

Verilog · seg\_decoder.v

```
module seg_decoder(input wire [3:0] bcd, output reg [7:0] seg_data);
    // [7:0] = {a,b,c,d,e,f,g,dp}; active high; hexadecimal 0..F; dp off.
    always @* begin
        case (bcd)
            0:seg_data=8'hfc; 1:seg_data=8'h60; 2:seg_data=8'hda; 3:seg_data=8'hf2;
            4:seg_data=8'h66; 5:seg_data=8'hb6; 6:seg_data=8'hbe; 7:seg_data=8'he0;
            8:seg_data=8'hfe; 9:seg_data=8'hf6; 10:seg_data=8'hee; 11:seg_data=8'h3e;
            12:seg_data=8'h9c; 13:seg_data=8'h7a; 14:seg_data=8'h9e; 15:seg_data=8'h8e;
            default:seg_data=0;
        endcase
    end
endmodule
```

### 3. Vivado Behavioral Simulation 결과

#### ✓ 완료 · 정상 동작

Tcl Console 출력: LAB1\_PASS seg\_decoder cases=16

검사 항목 **16개** 전부 통과, 시뮬레이션 종료 시각 **160ns** — 사전(VS Code·Icarus Verilog) 시뮬레이션과 동일한 결과였다.

### 4. 합성(Synthesis) 결과

#### ✓ 완료 · 정상 동작

Synthesis 오류 없이 완료. Report Utilization 기준 **LUT 4개**, **Bonded IOB 12개** 사용, Critical Warning 없음.

### 5. 구현(Implementation) · DRC 결과

#### ✓ 완료 · 정상 동작

Implementation 오류 없이 완료. Report DRC 기준 **위반(violation) 0건**. Spartan-7 बैं크 설정전압 경고를 막기 위해 XDC에 `CFGBVS / CONFIG_VOLTAGE` 속성을 지정했다.

### 6. 비트스트림 생성 및 보드 검증

#### ✓ 완료 · 정상 동작

Generate Bitstream 완료 후 Hardware Manager로 Spartan-7 (xc7s75fgga484-1) 탑재 HBE-Combo II-DLD 트레이너 보드에 프로그래밍했다. DIP 스위치 4개로 `bcd[3:0]`을 입력하고 7세그먼트 표시부에서 결과를 확인했다.

`bcd=0000` → “0”, `bcd=1000` → “8”, `bcd=1111` → “F” 세 경우를 DIP로 직접 입력해 7세그먼트에 올바른 숫자/문자가 표시되는 것을 확인했다. 실제 동작이 예상 진리표와 모두 일치했다.

## 7. 보드 동작 사진

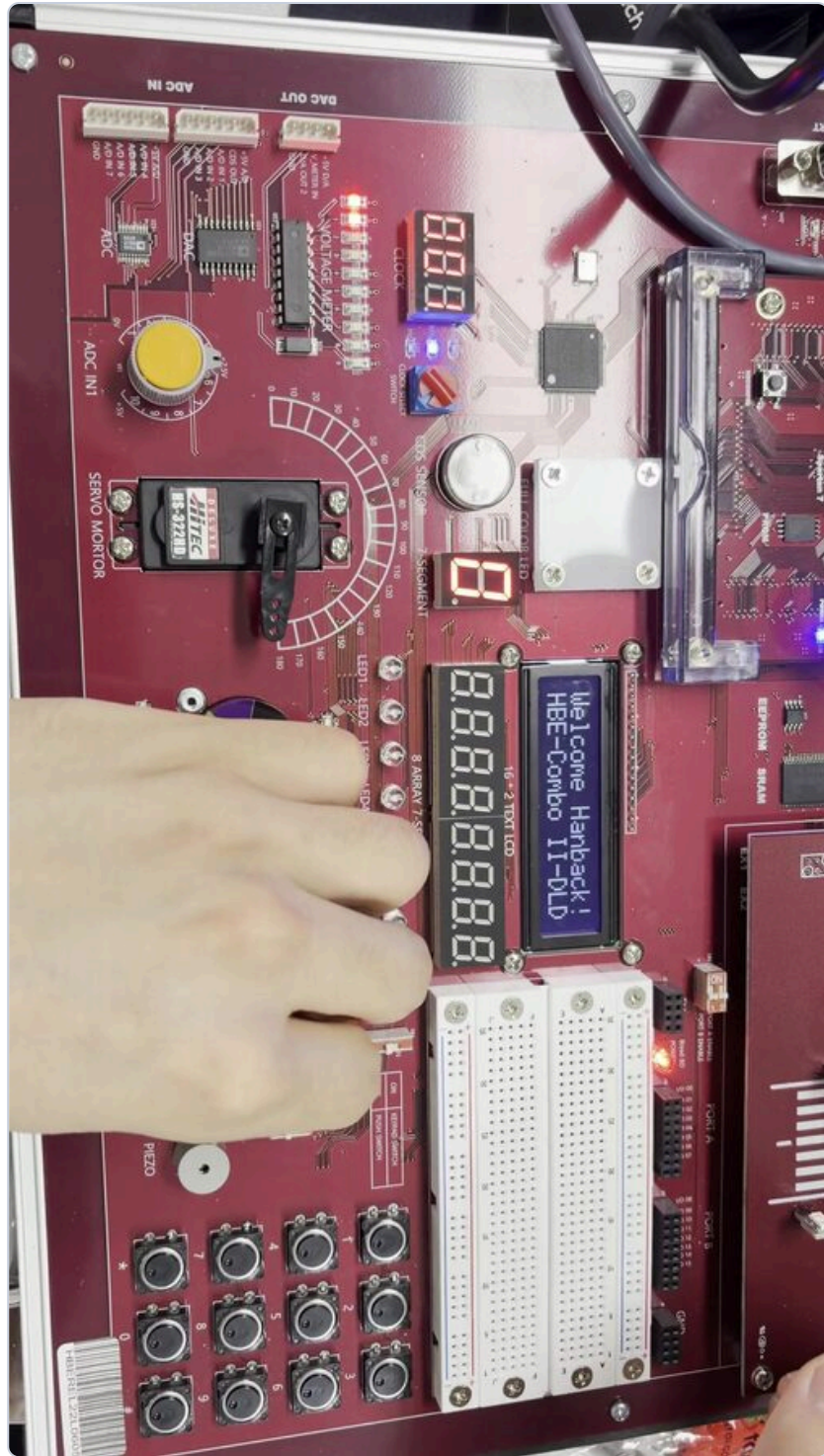


사진 10. LAB1-10 7세그먼트 디코더 (Seven-Segment Decoder) 보드 동작 확인 (DIP 스위치 4개로 bcd[3:0]을 입력하고 7세그먼트 표시부에서 결과를 확인했다.)

관련 영상: evidence/videos/LAB1-10\_\*.mp4 ([GitHub 저장소](#)에서 확인 가능)

## 실험 중 겪은 문제와 해결

### LAB1-01 등 초기 프로젝트

**증상:** Behavioral Simulation을 실행했더니 신호가 전부 미지(X/Z)로 나옴.

**원인 및 해결:** 테스트벤치(tb\_\*.sv) 파일이 Simulation Sources에 추가되지 않고 Top이 설계 모듈로 잡혀 있었다. Add Sources → "Add or create simulation sources"로 테스트벤치를 별도로 추가하고 Top을 다시 지정해 해결했다.

### LAB1-03, 05, 08 (256개 케이스)

**증상:** Behavioral Simulation이 1000ns에서 멈추고 PASS 문구가 나오지 않음.

**원인 및 해결:** 기본 시뮬레이션 실행시간(1000ns)이 2560ns 테스트벤치보다 짧았다. Tcl Console에 run -all을 입력해 \$finish까지 끝까지 실행되도록 해서 해결했다.

### LAB1-04

**증상:** Synthesis가 "Synth 8-6156 failed synthesizing 'tb\_sub\_4bit'" 오류로 실패함.

**원인 및 해결:** New Project 마법사에서 테스트벤치를 Design Sources와 함께 추가하는 바람에 합성 불가능한 테스트벤치가 Synthesis Top으로 잡혔다. 테스트벤치를 Design Sources에서 제거하고 sub\_4bit를 다시 Top으로 지정한 뒤, 테스트벤치는 Simulation Sources로만 추가해 해결했다.

### 전 회로 공통 (Spartan-7)

**증상:** DRC에서 बैंक 0 설정전압(CFGBVS) 관련 Critical Warning이 발생할 수 있음.

**원인 및 해결:** 모든 XDC 제약 파일에 set\_property CFGBVS VCCO [current\_design]과 set\_property CONFIG\_VOLTAGE 3.3 [current\_design]을 미리 추가해 예방했다.

### LAB1-10

**증상:** Program Device 단계에서 .bit 파일이 보이지 않음.

**원인 및 해결:** impl\_1 폴더를 확인한 결과 Implementation 리포트는 있었지만 Generate Bitstream이 실제로 완료되지 않은 상태였다. Generate Bitstream을 다시 실행해 비트스트림을 생성한 뒤 정상적으로 프로그래밍했다.

# 합성 자원 사용량 종합

회로별 LUT · Bonded IOB 사용량 (Spartan-7 xc7s75fgga484-1)

| 회로      | 모듈명         | LUT | Bonded IOB |
|---------|-------------|-----|------------|
| LAB1-01 | logic_gate  | 2   | 5          |
| LAB1-02 | full_adder  | 1   | 5          |
| LAB1-03 | adder_4bit  | 4   | 13         |
| LAB1-04 | sub_4bit    | 4   | 13         |
| LAB1-05 | compare_4   | 5   | 11         |
| LAB1-06 | mux_4x1     | 1   | 7          |
| LAB1-07 | demux_1x8   | 4   | 12         |
| LAB1-08 | encoder8x3  | 5   | 11         |
| LAB1-09 | decoder3x8  | 4   | 11         |
| LAB1-10 | seg_decoder | 4   | 12         |

10개 회로 모두 Critical Warning 없이 합성·구현이 완료되었고, DRC 위반은 0건이었다.

## 종합 결론 및 제출 체크리스트

10개 조합논리 회로를 RTL 설계 → 시뮬레이션 → 합성 → 구현 → 비트스트림 → 보드 검증까지 동일한 절차로 진행했고, 모든 단계에서 사전에 세운 예상 진리표와 일치하는 결과를 얻었다. 특히 다중 비트 입력을 갖는 회로(4비트 가산기·감산기·비교기·인코더 등)는 256가지 조합을 테스트벤치로 전수 검사해 설계의 신뢰성을 높였다.

- 10개 회로 모두 Vivado Behavioral Simulation 결과가 사전(VS Code) 결과와 일치함을 확인했다.
- 10개 회로 모두 Synthesis·Implementation을 오류 없이 완료했다.
- 10개 회로 모두 DRC 위반 0건을 확인했다.
- 10개 회로 모두 비트스트림을 생성하고 실제 보드에 프로그래밍했다.
- 10개 회로 모두 보드의 실제 입출력이 예상 진리표와 일치함을 사진으로 확인했다.